

Production GenAI & Data Science Systems

Chapter — Observability & Explainability in AI Systems

Making AI Decisions Transparent, Fair, and Trustworthy

Topics Covered

AI Explainability • LIME • SHAP • Fairness Engineering
Observability Pipelines • Production Monitoring • Bias Detection
SHAP Waterfall Plots • Fairlearn • AI Fairness 360

Preface — Why This Chapter Matters

As Artificial Intelligence becomes embedded into high-stakes systems — healthcare diagnostics, credit decisioning, hiring platforms, fraud detection, and autonomous agents — a fundamental question emerges: can humans trust AI decisions when they cannot see inside the machine?

Traditional software systems execute deterministic code that developers write explicitly. Every outcome is traceable to a line of logic. AI systems built on deep learning and Large Language Models operate very differently. They learn statistical patterns from data and make probabilistic decisions through neural network activations that are inherently opaque to human inspection.

This opacity creates what is commonly described as the Black Box problem. The model produces an output, but the reasoning pathway — which inputs drove the decision, why one class scored higher than another, and whether the model relied on biased features — remains hidden. In low-stakes settings this might be acceptable. In regulated industries and mission-critical applications, it is not.

This chapter addresses two disciplines that exist to solve the black-box problem: Explainability and Observability. We will study them from both a theoretical and a practical engineering standpoint, building real working implementations using industry-standard frameworks.

How This Chapter Is Organised

Section	What You Will Learn
Section 1 — Theory	Core concepts of Explainability, Observability, LIME, SHAP, and AI Fairness
Section 2 — Practice	Hands-on IPython notebook walkthroughs with full code explanations

SECTION 1

Theory & Conceptual Foundation

Chapter 1 — Introduction to AI Explainability

1.1 The Black-Box Problem

When an AI model rejects a loan application, recommends a medical treatment, or flags a transaction as fraudulent, the person affected by that decision deserves to know why it was made. Regulatory frameworks increasingly mandate this: the European Union's AI Act, GDPR's right to explanation, the US Equal Credit Opportunity Act, and many sector-specific regulations all require that consequential automated decisions be explainable.

The black-box problem is not merely technical. It is ethical, legal, and commercial. Organisations that cannot explain their AI decisions face regulatory fines, reputational damage, loss of user trust, and inability to debug model failures. Explainability is therefore a core engineering discipline, not an optional research curiosity.

1.2 What Is AI Explainability?

AI Explainability refers to the collection of methods, tools, and techniques that help humans understand how an AI system arrives at a particular decision or prediction. Explainability answers several critical questions:

- Why did the model produce this specific prediction?
- Which input features had the strongest influence on the result?
- Did any feature push the prediction in an unexpected direction?
- Was the decision affected by protected attributes such as race or gender?
- Can this prediction be trusted in the context it is being used?

Explainability is not the same as model transparency. A simple linear regression model is inherently transparent — you can read the coefficients and understand exactly what is happening. A 175-billion-parameter language model is not inherently transparent. Explainability frameworks provide post-hoc tools that create human-readable approximations of complex model behaviour.

1.3 The Business and Ethical Case for Explainability

The importance of explainability extends across multiple organisational dimensions:

Dimension	Why It Matters
Trust	Users and stakeholders trust systems whose decisions they can verify and understand.

Transparency	Decision-making becomes auditable by internal teams and external regulators.
Fairness	Bias in model reasoning becomes detectable and correctable.
Accountability	Organisations can attribute responsibility when systems produce incorrect outcomes.
Debugging	Engineers can identify which features or data patterns are causing model failures.
Compliance	Regulatory requirements for explainability can be satisfied systematically.
Model Improvement	Understanding what the model has learned guides better feature engineering and retraining.

1.4 Two Dimensions of Explainability

1.4.1 Local vs Global Explanations

A local explanation answers: why did the model make this specific prediction for this specific input? It provides granular insight into a single decision. A global explanation answers: what has the model learned overall? Which features matter most across the entire dataset? Both are valuable. Local explanations are more actionable for individual decisions; global explanations are more useful for understanding model behaviour at scale.

1.4.2 Model-Specific vs Model-Agnostic Methods

Model-specific methods exploit the internal structure of a particular architecture. For example, gradient-based attribution methods work only with neural networks that support backpropagation. Model-agnostic methods treat the model as a black box and observe how its outputs change in response to input perturbations. Model-agnostic methods are more versatile because they can be applied to any model type — a random forest, a gradient boosted tree, a transformer, or any other algorithm.

1.5 Explainability Frameworks Overview

The following table summarises the major explainability frameworks we will study in this chapter:

Framework	What It Does
LIME	Generates local, model-agnostic explanations by perturbing inputs and fitting a surrogate model.
SHAP	Computes mathematically grounded feature contributions using Shapley values from cooperative game theory.
Fairlearn	Assesses and mitigates fairness disparities across demographic groups (Microsoft).

AI Fairness 360	Detects, measures, and mitigates algorithmic bias using a comprehensive metrics library (IBM).
-----------------	--

Chapter 2 — LIME: Local Interpretable Model-Agnostic Explanations

2.1 Core Idea

LIME, introduced by Ribeiro et al. in 2016, solves the explainability problem by asking a deceptively simple question: even if I cannot understand the global model, can I understand it locally — in the immediate neighbourhood of this specific prediction? The answer is yes, because most complex functions, when examined over a small enough region, can be well-approximated by a much simpler linear function.

Think of it this way: the Earth's surface is a complex sphere, but if you stand in your garden and look around, the ground looks flat. LIME exploits this same principle — it creates a local flat approximation of the complex curved decision surface around any single prediction.

2.2 The Four Properties of LIME

Property	Explanation
Local	LIME explains one prediction at a time, not the entire model globally. This makes explanations specific and actionable.
Interpretable	The surrogate model LIME trains is a simple linear model or decision tree that humans can read and understand directly.
Model-Agnostic	LIME treats the original model as a black box. It works with neural networks, random forests, SVMs, transformers, and any other algorithm.
Explanations	LIME identifies which features (words, pixels, table columns) pushed the prediction up or down, and by how much.

2.3 How LIME Works — Step by Step

LIME follows a structured five-step process for generating any explanation:

1. Select the prediction to explain — a single input instance (a sentence, an image, a row of tabular data).
2. Generate perturbed samples — create many slightly modified versions of the input. For text, this means randomly removing words. For tabular data, it means sampling feature values from the training distribution.
3. Obtain predictions for each perturbed sample — pass all perturbed versions through the black-box model and record the output probabilities.
4. Weight samples by proximity — samples that are more similar to the original input are given higher weight, ensuring the surrogate model focuses on the local neighbourhood.

5. Train a simple surrogate model — fit an interpretable model (typically a linear regression or sparse linear model) on the weighted perturbed samples. The coefficients of this simple model reveal which features drive the prediction.

Key Insight: LIME does not explain the black-box model globally. It creates a local linear approximation that is faithful to the model's behaviour in the immediate neighbourhood of one specific prediction. The explanation may not be globally accurate, but it is locally faithful.

2.4 A Real-World Example — Sentiment Analysis

Consider a sentiment classification model built with DistilBERT that analyses restaurant reviews. A user submits the review: "I didn't feel like an antiques store, more like a garage sale." The model predicts Negative sentiment with 87% confidence. But why?

LIME generates thousands of perturbed versions of this sentence — versions with words randomly masked out — and observes how the model's confidence score changes. It discovers that:

- The words 'garage sale' strongly push the prediction toward Negative.
- The phrase 'antiques store' has a mild positive contribution.
- The word 'didn't' has a strong negative contribution by inverting the sentiment of the surrounding context.

This word-level attribution is the explanation LIME produces. An engineer reviewing this explanation might notice whether the model is relying on sensible linguistic patterns or exploiting spurious correlations in the training data.

2.5 Where LIME Is Used in Production

- Sentiment analysis — understanding why reviews are classified as positive or negative.
- Fraud detection — explaining which transaction features triggered a fraud flag.
- Loan approval systems — providing regulators with per-decision explanations.
- Healthcare diagnostics — explaining which clinical features influenced a diagnostic prediction.
- Recommendation systems — understanding why a particular item was recommended.

Chapter 3 — SHAP: SHapley Additive exPlanations

3.1 The Mathematical Foundation — Cooperative Game Theory

SHAP, developed by Lundberg and Lee in 2017, brings mathematical rigour to explainability by grounding feature attribution in Shapley values from cooperative game theory. The Shapley value concept originates from economics: in a cooperative game where multiple players collaborate to generate a reward, how should that reward be fairly distributed among the players, given that each player contributed differently?

SHAP translates this concept directly into machine learning. The 'players' are the input features. The 'reward' is the model's prediction. Each feature receives a Shapley value that represents its fair contribution to moving the prediction away from the average baseline prediction. The mathematics guarantee that this attribution is unique, efficient, and axiomatically fair.

3.2 The SHAP Decomposition Formula

SHAP represents any model prediction as a linear sum of feature contributions:

$$\text{Prediction} = \text{Base Value} + \Phi_1(\text{feature}_1) + \Phi_2(\text{feature}_2) + \dots + \Phi_n(\text{feature}_n)$$

Where:

- Base Value is the average prediction across the training dataset — what the model would predict if it knew nothing about this specific instance.
- Φ_i (phi-i) is the Shapley value for feature i — the amount that feature i pushes the prediction above or below the base value.
- The sum of all Shapley values plus the base value equals the actual prediction exactly — there is no unexplained residual.

3.3 A Concrete Example — House Price Prediction

Suppose a model predicts house prices. The average house price in the training data is £300,000. For a specific house, the model predicts £420,000. SHAP explains the difference of £120,000 as follows:

Feature	SHAP Contribution
Number of Bedrooms (5)	+£65,000 — pulls price up strongly
Location (Zone 2, London)	+£45,000 — pulls price up
Square Footage (1,800 sq ft)	+£25,000 — pulls price up
Age of Property (1970s)	-£15,000 — pulls price down

Total	+£120,000 above base value of £300,000
-------	--

This decomposition is transparent, quantitative, and auditable. Every pound of the predicted price is accounted for by a specific feature.

3.4 SHAP Visualisations

3.4.1 Waterfall Plots

A waterfall plot shows how each feature contribution sequentially builds up the final prediction from the base value. Features that push the prediction upward are shown in red (positive contributions). Features that pull the prediction downward are shown in blue (negative contributions). The plot starts at the base value and ends at the final prediction, with each bar representing one feature's Shapley value.

3.4.2 Summary Plots (Global Explanations)

While waterfall plots are local (per prediction), SHAP summary plots aggregate Shapley values across many predictions to reveal global patterns. They show which features are most important overall and how the direction of each feature's effect varies with its value. For example, a summary plot might reveal that high income strongly increases loan approval probability, while previous defaults strongly decrease it.

3.5 LIME vs SHAP — A Detailed Comparison

Property	LIME	SHAP
Scope	Local only — one prediction at a time	Local and Global — supports both levels
Theoretical Foundation	Surrogate model approximation (heuristic)	Shapley values from cooperative game theory (axiomatic)
Mathematical Guarantee	No formal guarantee of faithfulness	Guaranteed efficiency, symmetry, and linearity
Speed	Faster — surrogate training is lightweight	Slower — Shapley value computation is expensive
Stability	Less consistent — results vary across runs	Highly consistent — mathematically determined values
Ease of Interpretation	Intuitive word/feature lists	Rich quantitative decompositions with precise magnitudes
Best Use Case	Quick local explanations, NLP tasks	Rigorous analysis, model debugging, regulatory compliance

Chapter 4 — AI Fairness and Responsible AI

4.1 Why AI Systems Can Be Unfair

AI models learn from historical data. If that historical data reflects past human discrimination — hiring practices that favoured certain demographics, lending policies that redlined certain neighbourhoods, criminal justice systems with racial disparities — the model will learn and reproduce those discriminatory patterns. The model is not inherently malicious; it is doing exactly what it was designed to do: minimise prediction error on the training data. But if the training data encodes historical injustice, the model perpetuates it at scale.

This creates what researchers call algorithmic bias. Unlike human bias, which occurs one decision at a time, algorithmic bias operates at machine speed across millions of decisions, amplifying historical inequalities systematically.

4.2 Types of Fairness

- Demographic parity — the model should produce positive outcomes at equal rates across demographic groups.
- Equalised odds — the model's true positive rate and false positive rate should be equal across groups.
- Individual fairness — similar individuals should receive similar predictions, regardless of group membership.
- Counterfactual fairness — if an individual's protected attribute were changed while keeping everything else the same, the prediction should not change.

4.3 AI Fairness 360 (AIF360)

IBM's AI Fairness 360 is an open-source Python toolkit that provides a comprehensive collection of fairness metrics and bias mitigation algorithms. It enables data scientists to audit datasets and models for bias before deployment and to apply correction techniques during preprocessing, in-processing (during model training), or post-processing (after predictions are made).

- Pre-processing algorithms — modify training data to remove discriminatory patterns before model training.
- In-processing algorithms — apply fairness constraints during the model training optimisation process.
- Post-processing algorithms — adjust model predictions after inference to achieve fairness targets.

4.4 Fairlearn (Microsoft)

Microsoft's Fairlearn library focuses on visualisation and mitigation of fairness problems in classification and regression models. It provides an interactive dashboard that compares model

performance across demographic groups, making it straightforward to identify which groups are being systematically disadvantaged.

Fairlearn's mitigation algorithms — including Exponentiated Gradient and Grid Search — allow engineers to trade off overall accuracy for improved fairness across groups. In regulated industries such as financial services and healthcare, this trade-off is not optional — regulators mandate it.

Chapter 5 — Observability in Production AI Systems

5.1 Explainability vs Observability — A Critical Distinction

These two concepts are related but address different questions. Explainability asks: why did the model make this specific prediction? It is primarily concerned with interpreting individual decisions. Observability asks: how is the system behaving in production over time? It is primarily concerned with monitoring the overall health, reliability, and safety of the deployed system.

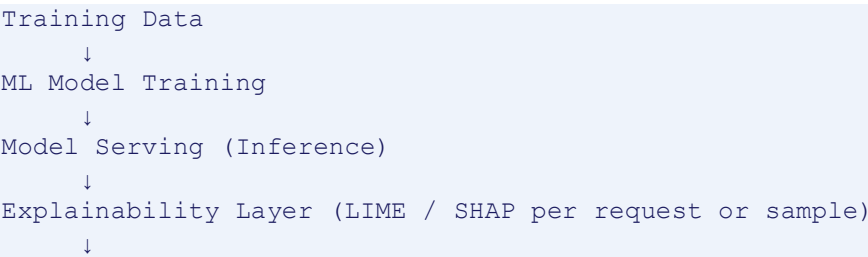
Concept	Primary Question	When Applied
Explainability	Why did this specific prediction happen?	At inference time — per prediction
Observability	How is the overall system performing?	Continuously — across all predictions over time

5.2 What AI Observability Monitors

- Latency — how long does each inference request take? Are response times degrading?
- Data drift — are the statistical properties of incoming requests shifting away from the training distribution?
- Concept drift — is the relationship between input features and the correct output changing over time?
- Hallucination tracking — for language models, are responses containing factually incorrect or fabricated information?
- Token usage monitoring — what is the per-request cost in tokens, and is it within budget?
- Failure analysis — which request types are producing errors or poor-quality outputs?
- Infrastructure metrics — CPU, GPU, memory utilisation, queue depths, and scaling behaviour.

5.3 The Production Explainability Pipeline

In a production system, explainability is not run manually on individual predictions. It is embedded into an automated pipeline that continuously monitors model behaviour:



```
Fairness Monitoring (Fairlearn / AIF360)
  ↓
Drift Detection (statistical tests on feature distributions)
  ↓
Alerting & Dashboards (Grafana, MLflow, custom)
  ↓
Human Review & Governance
```

This pipeline ensures that model behaviour is continuously scrutinised, deviations are detected early, and human reviewers are alerted before failures impact users at scale.

5.4 Why Observability Is Non-Negotiable in Production

Models degrade. The world changes. User behaviour evolves. Supply chains shift. Medical knowledge advances. Any model trained at a point in time will gradually become less accurate as the world it was trained to represent diverges from the world it is now operating in. Without observability, this degradation is invisible until it causes a significant failure. With observability, engineers can detect it early, retrain, and redeploy before users are harmed.

SECTION 2

Practical Implementation & IPython Notebook Walkthrough

Chapter 6 — Implementing LIME for Text Explainability

6.1 Prerequisites and Project Context

In this walkthrough we build an end-to-end explainability system for a text sentiment classifier. We use DistilBERT — a lightweight, production-grade transformer model — as our black-box classifier, and LIME to generate human-readable explanations for its predictions. By the end of this section you will be able to apply LIME to any text classification model in production.

Cell 1 — Installing Dependencies

```
# Cell 1: Install all required libraries
!pip install lime shap transformers scikit-learn matplotlib torch
```

What this cell does and why each library is needed:

Library	Purpose in This Project
lime	The core LIME explainability framework. Provides LimeTextExplainer for NLP tasks.
shap	The SHAP explainability framework for Shapley-value-based feature attribution.
transformers	Hugging Face Transformers library — provides access to DistilBERT and other pre-trained models.
scikit-learn	Machine learning utilities for data preparation, metrics, and pipeline integration.
matplotlib	Plotting library for visualising explanations, feature importance charts, and SHAP plots.
torch	PyTorch — the deep learning backend for running DistilBERT inference.

Run this cell once at the beginning of your notebook session. In a production environment, these dependencies should be pinned in a requirements.txt file with exact version numbers to ensure reproducibility.

Cell 2 — Importing Libraries

```
# Cell 2: Import all required modules
from lime.lime_text import LimeTextExplainer
from transformers import pipeline
import numpy as np
import matplotlib.pyplot as plt
```

Detailed explanation of each import:

LimeTextExplainer — This is the primary class from the LIME library for explaining text classification models. It is designed specifically for NLP tasks and handles text-specific perturbation strategies (word removal/replacement). It is distinct from LimeTabularExplainer (for structured data) and LimeImageExplainer (for images).

from transformers import pipeline — The Hugging Face pipeline function is a high-level API that wraps a pre-trained model with its tokenizer into a single callable object. We use it to create our sentiment classifier without having to manually manage tokenisation, tensor conversion, and softmax computation.

numpy — Provides numerical array operations. LIME internally expects the prediction function to return a NumPy array of class probabilities.

Cell 3 — Loading the Pre-trained Sentiment Classifier

```
# Cell 3: Load a pre-trained sentiment analysis model
sentiment_pipeline = pipeline(
    task='text-classification',
    model='distilbert-base-uncased-finetuned-sst-2-english',
    return_all_scores=True
)

# Quick sanity check
test_result = sentiment_pipeline('This movie was absolutely fantastic!')
print(test_result)
```

What is happening in this cell:

pipeline(task='text-classification', ...) — Creates a complete inference pipeline. Under the hood, Hugging Face downloads the model weights and tokenizer configuration from its model hub (approximately 260MB for DistilBERT).

model='distilbert-base-uncased-finetuned-sst-2-english' — Specifies DistilBERT fine-tuned on the Stanford Sentiment Treebank dataset (SST-2). This model classifies text as POSITIVE or NEGATIVE with high accuracy on English sentiment tasks.

return_all_scores=True — Critical parameter. By default the pipeline returns only the winning class. We need probabilities for all classes (NEGATIVE and POSITIVE) because LIME requires the prediction function to return a full probability distribution over all output classes.

Cell 4 — Wrapping the Model for LIME Compatibility

```
# Cell 4: Create a LIME-compatible prediction function
def predict_probabilities(texts):
    """
    LIME requires a prediction function that:
    - Accepts a list of strings
    - Returns a 2D numpy array of shape (n_samples, n_classes)
    """
    results = sentiment_pipeline(texts)
    # Extract probabilities for each class in consistent order
    probs = np.array([
        [
            result[0]['score'] if result[0]['label'] == 'NEGATIVE' else
            result[1]['score'],
            result[0]['score'] if result[0]['label'] == 'POSITIVE' else
            result[1]['score'],
        ]
        for result in results
    ])
    return probs
```

Why this wrapper function is necessary:

LIME's `explain_instance()` method requires a prediction function with a very specific signature. It must accept a Python list of strings (the perturbed text samples LIME generates) and return a 2D NumPy array where each row is a probability distribution over all output classes. The Hugging Face pipeline does not return results in this format by default — it returns a list of dictionaries. This wrapper translates between the two formats.

The order of classes in the returned array must be consistent and must match the `class_names` argument you pass to `LimeTextExplainer`. In this implementation we always place NEGATIVE probability first (index 0) and POSITIVE probability second (index 1).

Cell 5 — Initialising the LIME Explainer


```
# Cell 5: Create the LIME text explainer
explainer = LimeTextExplainer(
    class_names=['Negative', 'Positive']
)
```

LimeTextExplainer — Creates the core LIME engine for text data. The constructor configures how LIME will perturb inputs (by randomly masking words with a placeholder token) and how it will weight samples (using an exponential kernel based on cosine similarity).

class_names=['Negative', 'Positive'] — Provides human-readable labels for the output classes. These labels appear in the explanation visualisations. Without class_names, LIME would display 'Class 0' and 'Class 1', which are meaningless to end users. The order must exactly match the column order of the array returned by your prediction function.

Cell 6 — Generating a LIME Explanation

```
# Cell 6: Generate explanation for a specific review
review_text = "I didn't feel like an antiques store, more like a garage
sale."

explanation = explainer.explain_instance(
    text_instance=review_text,      # The text to explain
    classifier_fn=predict_probabilities, # Your wrapped prediction function
    num_features=6,                # Show top 6 most influential words
    num_samples=1000,              # Generate 1000 perturbed variants
    labels=[1]                     # Explain the Positive class
)

# Print the ranked feature explanations
print('Explanation for class: Positive')
for feature, weight in explanation.as_list(label=1):
    direction = 'INCREASES' if weight > 0 else 'DECREASES'
    print(f"  '{feature}' {direction} Positive probability by
{abs(weight):.4f}")
```

Detailed parameter breakdown:

Parameter	What It Controls
text_instance	The specific text sample whose prediction you want to explain. This is the single data point being examined.
classifier_fn	Your wrapped prediction function. LIME will call this function hundreds of times with perturbed versions of text_instance.

num_features	The maximum number of words (features) to include in the explanation. Higher values give more complete explanations but can be harder to read. 6 is a good starting point for most use cases.
num_samples	How many perturbed text variants LIME generates. Higher values produce more stable and accurate explanations at the cost of more inference calls to your model. 1000 is a reasonable default; increase to 3000+ for production explanations.
labels	Which output class to generate explanations for. labels=[1] means explain the Positive class. You can pass multiple labels to get explanations for all classes simultaneously.

Cell 7 — Visualising the LIME Explanation

```
# Cell 7: Visualise the explanation

# Option 1: Show as styled HTML (works in Jupyter notebooks)
explanation.show_in_notebook(text=True)

# Option 2: Save as a standalone HTML file
explanation.save_to_file('lime_explanation.html')

# Option 3: Get matplotlib figure for embedding in reports
fig = explanation.as_pyplot_figure(label=1)
plt.title('LIME Feature Importances - Positive Sentiment', fontsize=14)
plt.tight_layout()
plt.savefig('lime_explanation.png', dpi=150, bbox_inches='tight')
plt.show()
```

LIME provides three rendering options depending on your use case:

- `show_in_notebook()` — renders an interactive colour-coded HTML widget directly in the Jupyter notebook output. Words highlighted in green increase the predicted class probability; words highlighted in red decrease it. This is the most user-friendly format for exploratory analysis.
- `save_to_file()` — saves the HTML explanation to disk. Useful for sharing explanations with non-technical stakeholders via email or web browser.
- `as_pyplot_figure()` — returns a standard matplotlib Figure object showing a horizontal bar chart of feature importances. This format is ideal for embedding in automated reports or dashboards.

Chapter 7 — Implementing SHAP for Tabular Data

7.1 Project Context

In this walkthrough we apply SHAP to a tabular machine learning task — predicting whether a loan application should be approved. We train a gradient boosted classifier, then use SHAP to generate both local explanations (per application) and global explanations (model behaviour overall). This is a realistic representation of how explainability is applied in regulated financial services environments.

Cell 8 — Loading Libraries and Data

```
# Cell 8: Import SHAP and supporting libraries
import shap
import pandas as pd
import numpy as np
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.model_selection import train_test_split
from sklearn.datasets import fetch_california_housing
import matplotlib.pyplot as plt

# Load dataset (using California housing as a proxy regression example)
data = fetch_california_housing(as_frame=True)
X, y = data.data, data.target

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print(f'Training samples: {X_train.shape[0]}')
print(f'Test samples:      {X_test.shape[0]}')
print(f'Features:          {list(X.columns)}')
```

import shap — Imports the SHAP library. SHAP provides multiple explainer classes optimised for different model types: `TreeExplainer` (for tree-based models), `DeepExplainer` (for neural networks), `LinearExplainer` (for linear models), and the general-purpose `KernelExplainer` (for any model, but slower).

train_test_split(..., random_state=42) — The `random_state` parameter ensures reproducibility. Every run of the notebook with the same `random_state` will produce the same train/test split, which is critical for scientific reproducibility and debugging.

Cell 9 — Training the Model

```
# Cell 9: Train a Gradient Boosting model
model = GradientBoostingClassifier(
    n_estimators=100,
    max_depth=4,
    learning_rate=0.1,
    random_state=42
)
model.fit(X_train, y_train)

print(f'Training complete. R² Score: {model.score(X_test, y_test):.3f}')
```

Gradient Boosted Trees are among the most popular models in tabular ML competitions and production deployments. They build an ensemble of decision trees sequentially, where each tree corrects the errors of the previous ones. Key hyperparameters:

Hyperparameter	Effect
n_estimators=100	Number of trees in the ensemble. More trees improve accuracy up to a point, then plateau.
max_depth=4	Maximum depth of each tree. Deeper trees can capture more complex patterns but risk overfitting.
learning_rate=0.1	Shrinkage factor applied to each tree's contribution. Lower values require more trees but generalise better.

Cell 10 — Creating the SHAP Explainer

```
# Cell 10: Create SHAP TreeExplainer
explainer = shap.TreeExplainer(model)

# Compute SHAP values for the test set
shap_values = explainer(X_test)

print(f'SHAP values shape: {shap_values.values.shape}')
print(f'Base value (average prediction): {explainer.expected_value:.3f}')
```

shap.TreeExplainer(model) — Creates a SHAP explainer specifically optimised for tree-based models. TreeExplainer uses an exact, polynomial-time algorithm to compute Shapley values by exploiting the structure of decision trees. It is significantly faster than KernelExplainer, which uses sampling-based approximation.

explainer(X_test) — Computes SHAP values for every row in X_test. The returned object (shap_values) is a SHAP Explanation object containing:

- `shap_values.values` — a matrix of shape `(n_samples, n_features)` containing each feature's Shapley contribution for each prediction.
- `shap_values.base_values` — the baseline prediction (average model output), identical for all samples.
- `shap_values.data` — the actual feature values for each sample.

Cell 11 — Waterfall Plot (Local Explanation)

```
# Cell 11: Generate waterfall plot for a single prediction
sample_index = 0 # Explain the first test sample

plt.figure(figsize=(10, 6))
shap.plots.waterfall(
    shap_values[sample_index],
    max_display=10,          # Show top 10 most impactful features
    show=False
)
plt.title(f'SHAP Waterfall - Sample {sample_index}', fontsize=14,
fontweight='bold')
plt.tight_layout()
plt.savefig('shap_waterfall.png', dpi=150, bbox_inches='tight')
plt.show()
```

Reading a waterfall plot: The x-axis represents prediction units (e.g., house price in hundreds of thousands of dollars). The plot starts at $E[f(X)]$ — the average model prediction across the training data. Each horizontal bar then shows one feature's contribution:

- Red bars (pointing right) — this feature's value for this sample pushes the prediction above the baseline.
- Blue bars (pointing left) — this feature's value for this sample pulls the prediction below the baseline.
- The final bar ends at $f(x)$ — the actual prediction for this specific sample.

`max_display=10` — Limits the plot to showing only the 10 features with the largest absolute SHAP values. Remaining features are grouped into a single 'other features' bar to keep the visualisation readable.

Cell 12 — Summary Plot (Global Explanation)

```
# Cell 12: Generate summary plot for global model behaviour
plt.figure(figsize=(10, 7))
shap.summary_plot(
    shap_values.values,      # SHAP value matrix: (n_samples, n_features)
    X_test,                  # Original feature values for colour mapping
    feature_names=list(X.columns),
```

```
        show=False
    )
    plt.title('SHAP Summary – Global Feature Importance', fontsize=14,
              fontweight='bold')
    plt.tight_layout()
    plt.savefig('shap_summary.png', dpi=150, bbox_inches='tight')
    plt.show()
```

The summary plot is one of the most powerful tools in the SHAP toolkit for understanding model behaviour at a global level. Each row represents one feature. Within each row:

- Each dot represents one data sample from `X_test`.
- The horizontal position of the dot shows the SHAP value — how much that feature pushed the prediction left (negative, below average) or right (positive, above average) for that sample.
- The colour of the dot shows the actual feature value for that sample — red for high feature values, blue for low feature values.

Reading the plot: if red dots (high feature value) appear on the right side (positive SHAP), it means that high values of that feature increase the prediction. If red dots appear on the left (negative SHAP), high feature values decrease the prediction. This reveals the directionality of each feature's influence.

Cell 13 — SHAP Feature Importance Bar Chart

```
# Cell 13: Bar chart of mean absolute SHAP values
plt.figure(figsize=(10, 6))
shap.summary_plot(
    shap_values.values,
    X_test,
    feature_names=list(X.columns),
    plot_type='bar',      # Switch to bar chart mode
    show=False
)
plt.title('Mean |SHAP Value| – Feature Importance Ranking', fontsize=14,
          fontweight='bold')
plt.tight_layout()
plt.savefig('shap_bar.png', dpi=150, bbox_inches='tight')
plt.show()
```

The bar chart version of the summary plot collapses each feature's SHAP value distribution into a single number: the mean absolute SHAP value across all samples in `X_test`. This provides a clean, ranked list of global feature importance — equivalent to the feature importances you would get from `model.feature_importances_`, but grounded in Shapley value theory and therefore more statistically principled.

Chapter 8 — Fairness Analysis with Fairlearn

8.1 Project Context

In this walkthrough we audit a hiring prediction model for demographic bias using Microsoft's Fairlearn library. We train a classifier that predicts whether a candidate should be shortlisted for an interview, then measure whether the model's predictions are equally fair across gender groups. This mirrors real-world compliance requirements in HR and recruitment AI systems.

Cell 14 — Installing Fairlearn

```
# Cell 14: Install the Fairlearn library
!pip install fairlearn
```

Fairlearn is maintained by Microsoft and is actively developed as of 2025. It integrates with scikit-learn models and provides both metrics for measuring unfairness and algorithms for mitigating it. Install this in your requirements.txt with a pinned version for production use.

Cell 15 — Importing Fairlearn Components

```
# Cell 15: Import Fairlearn components
from fairlearn.metrics import (
    MetricFrame,
    selection_rate,
    demographic_parity_difference,
    equalized_odds_difference
)
from fairlearn.reductions import ExponentiatedGradient, DemographicParity
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
import pandas as pd
import numpy as np
```

Key components imported and their roles:

Component	Purpose
MetricFrame	Core Fairlearn class for computing metrics disaggregated by sensitive feature groups. Think of it as a fairness-aware version of sklearn's classification_report.
selection_rate	Measures what fraction of individuals in each group received a positive prediction. Equal selection rates across groups = demographic parity.

demographic_parity_difference	Computes the gap between the highest and lowest selection rates across groups. A value of 0 means perfect demographic parity.
equalized_odds_difference	Measures the maximum difference in true positive rate or false positive rate across groups. A value of 0 means equalised odds.
ExponentiatedGradient	A fairness-constrained learning algorithm that iteratively adjusts the model to satisfy a specified fairness constraint.
DemographicParity	The fairness constraint class — tells ExponentiatedGradient to enforce equal selection rates across groups.

Cell 16 — Preparing a Synthetic Dataset

```
# Cell 16: Create a synthetic hiring dataset with known bias
np.random.seed(42)
n_samples = 1000

# Simulate features: years of experience, test score, degree level
experience = np.random.normal(5, 3, n_samples).clip(0, 20)
test_score = np.random.normal(70, 15, n_samples).clip(0, 100)
degree = np.random.choice([0, 1, 2], n_samples) # 0=none, 1=bachelor, 2=master+
gender = np.random.choice(['Male', 'Female'], n_samples) # Sensitive attribute

# Introduce bias: add +5 to test score for Male candidates (simulates historical bias)
biased_score = test_score + (5 * (gender == 'Male'))

# Generate label: 1 = shortlisted
shortlisted = ((experience > 4) & (biased_score > 72) & (degree >= 1)).astype(int)

# Build dataframe
df = pd.DataFrame({
    'experience': experience,
    'test_score': test_score,
    'degree': degree,
    'gender': gender,
    'shortlisted': shortlisted
})

print(f'Overall shortlist rate: {shortlisted.mean():.2%}')
print(df.groupby('gender')['shortlisted'].mean())
```

This cell deliberately introduces a known bias: male candidates receive a five-point bonus to their test score during label generation, simulating a hiring process that historically favoured one gender. This is a controlled experiment — we know exactly what the bias is, so we can verify whether Fairlearn detects and mitigates it correctly.

Cell 17 — Training a Biased Model and Measuring Unfairness

```
# Cell 17: Train a standard model and measure its fairness
from sklearn.model_selection import train_test_split

features = ['experience', 'test_score', 'degree']
X = df[features]
y = df['shortlisted']
sensitive = df['gender']    # The sensitive attribute - NOT used as a feature

X_train, X_test, y_train, y_test, s_train, s_test = train_test_split(
    X, y, sensitive, test_size=0.3, random_state=42
)

# Train unconstrained model
base_model = LogisticRegression(random_state=42)
base_model.fit(X_train, y_train)
y_pred = base_model.predict(X_test)

# Compute fairness metrics
mf = MetricFrame(
    metrics={'selection_rate': selection_rate, 'accuracy': accuracy_score},
    y_true=y_test,
    y_pred=y_pred,
    sensitive_features=s_test
)

print('=== Biased Model - Metrics by Group ===')
print(mf.by_group)
print(f'\nDemographic Parity Difference:
{demographic_parity_difference(y_test, y_pred,
sensitive_features=s_test):.4f}')
```

MetricFrame(...) — This is Fairlearn's core fairness measurement object. It computes any scikit-learn compatible metric function disaggregated by groups in the `sensitive_features` column. The result is a DataFrame showing each metric for each group, making disparities immediately visible.

demographic_parity_difference — Returns the difference between the maximum and minimum selection rate across groups. A value of 0.15 means the group with the highest selection rate is 15 percentage points above the group with the lowest. Most regulatory frameworks consider disparities above 0.10–0.15 as requiring remediation.

Cell 18 — Applying Fairness-Constrained Learning

```
# Cell 18: Mitigate bias using ExponentiatedGradient
from fairlearn.reductions import ExponentiatedGradient, DemographicParity

# Define the fairness constraint
constraint = DemographicParity()

# Create the fairness-constrained learner
mitigator = ExponentiatedGradient(
    estimator=LogisticRegression(random_state=42),
    constraints=constraint,
    eps=0.05 # Allowed fairness violation tolerance
)

# Train with fairness constraint
mitigator.fit(X_train, y_train, sensitive_features=s_train)
y_pred_fair = mitigator.predict(X_test)

# Re-measure fairness
mf_fair = MetricFrame(
    metrics={'selection_rate': selection_rate, 'accuracy': accuracy_score},
    y_true=y_test,
    y_pred=y_pred_fair,
    sensitive_features=s_test
)

print('=== Fair Model - Metrics by Group ===')
print(mf_fair.by_group)
print(f'\nDemographic Parity Difference after mitigation:
{demographic_parity_difference(y_test, y_pred_fair,
sensitive_features=s_test):.4f}')
```

ExponentiatedGradient — This is a meta-algorithm that wraps any scikit-learn compatible estimator (here, `LogisticRegression`) and trains it subject to a fairness constraint. It works by solving a minimax optimisation problem: minimise prediction error subject to the constraint that the specified fairness metric does not exceed the tolerance `eps`.

eps=0.05 — The allowed fairness violation tolerance. Setting `eps=0.05` tells `ExponentiatedGradient` to produce a model where the demographic parity difference is at most 5 percentage points. Smaller `eps` values enforce stricter fairness but typically reduce overall accuracy. This is the fundamental fairness-accuracy trade-off.

Cell 19 — Visualising the Fairness-Accuracy Trade-off

```
# Cell 19: Compare biased vs fair model side by side
comparison = pd.DataFrame({
    'Model': ['Biased Model', 'Fair Model'],
    'Overall Accuracy': [
```

```
        accuracy_score(y_test, y_pred),
        accuracy_score(y_test, y_pred_fair)
    ],
    'Demographic Parity Difference': [
        demographic_parity_difference(y_test, y_pred,
sensitive_features=s_test),
        demographic_parity_difference(y_test, y_pred_fair,
sensitive_features=s_test)
    ]
})

print(comparison.to_string(index=False))

# Visualise
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

for ax, col, title, color in zip(
    axes,
    ['Overall Accuracy', 'Demographic Parity Difference'],
    ['Accuracy (Higher is Better)', 'Fairness Gap (Lower is Better)'],
    ['steelblue', 'darkorange']
):
    ax.bar(comparison['Model'], comparison[col], color=color, alpha=0.8)
    ax.set_title(title, fontsize=12, fontweight='bold')
    ax.set_ylim(0, max(comparison[col]) * 1.3)
    for i, v in enumerate(comparison[col]):
        ax.text(i, v + 0.01, f'{v:.3f}', ha='center', fontweight='bold')

plt.suptitle('Biased vs Fair Model – Accuracy-Fairness Trade-off',
    fontsize=14, fontweight='bold')
plt.tight_layout()
plt.savefig('fairness_comparison.png', dpi=150, bbox_inches='tight')
plt.show()
```

This comparison visualisation is what you would present to regulators, compliance teams, or ethics boards to demonstrate that fairness analysis was performed and bias mitigation was applied. It quantitatively shows both the fairness improvement (reduced demographic parity difference) and the accuracy cost (if any) of the mitigation.

Chapter 9 — Building a Production Observability Pipeline

9.1 The Goal

In this final chapter we assemble the individual components — SHAP explanations, fairness monitoring, and performance tracking — into a unified observability pipeline that can run continuously in production. This is the architecture that real enterprise AI systems use to maintain safety, reliability, and governance over time.

Cell 20 — Logging Predictions with SHAP Explanations

```
# Cell 20: Production prediction logger with built-in explainability
import json
import datetime

def log_prediction_with_explanation(model, explainer, X_instance,
feature_names):
    """
    Makes a prediction, generates a SHAP explanation,
    and returns a structured log entry for auditing.
    """
    # Generate prediction
    prediction = model.predict(X_instance)[0]
    probability = model.predict_proba(X_instance)[0].max()

    # Generate SHAP explanation
    shap_vals = explainer(X_instance)
    top_features = sorted(
        zip(feature_names, shap_vals.values[0]),
        key=lambda x: abs(x[1]),
        reverse=True
   )[:5] # Top 5 most influential features

    # Structured log entry
    log_entry = {
        'timestamp': datetime.datetime.utcnow().isoformat(),
        'prediction': int(prediction),
        'confidence': float(probability),
        'top_feature_contributions': [
            {'feature': f, 'shap_value': float(v)} for f, v in top_features
        ]
    }

    return log_entry

# Example usage
```

```
sample = X_test.iloc[[0]]
log = log_prediction_with_explanation(base_model, explainer, sample,
features)
print(json.dumps(log, indent=2))
```

This logging function is the core of an explainable production system. Every prediction is stored not just as a number, but with a full explanation of which features drove it. This log can be stored in a database, streamed to a monitoring platform like MLflow or Weights & Biases, or fed into an audit system for regulatory compliance review.

Cell 21 — Drift Detection

```
# Cell 21: Simple feature drift detection using statistical tests
from scipy import stats

def detect_data_drift(reference_data, current_data, feature_names,
alpha=0.05):
    """
    Compares statistical distributions of reference (training) data
    against current (production) data using the Kolmogorov-Smirnov test.
    Flags features where the distribution has significantly shifted.
    """
    drift_report = {}
    for feature in feature_names:
        stat, p_value = stats.ks_2samp(
            reference_data[feature],
            current_data[feature]
        )
        drift_report[feature] = {
            'ks_statistic': float(stat),
            'p_value': float(p_value),
            'drift_detected': bool(p_value < alpha)
        }
    return drift_report

# Simulate production data with mild drift on experience feature
production_sample = X_test.copy()
production_sample['experience'] = production_sample['experience'] * 1.3 #
Shift distribution

drift_report = detect_data_drift(X_train, production_sample, features)
for feat, report in drift_report.items():
    status = 'DRIFT DETECTED' if report['drift_detected'] else 'Stable'
    print(f'{feat:15s}: p={report["p_value"]:.4f} [{status}']')
```

stats.ks_2samp — The Kolmogorov-Smirnov two-sample test is a non-parametric statistical test that compares two probability distributions. It measures the maximum difference between the cumulative distribution functions of the reference and current data. A low p-value (below alpha,

typically 0.05) indicates that the two distributions are statistically significantly different — meaning drift has occurred.

In production, this function would run automatically every time a batch of new predictions is collected. If drift is detected in critical features, an alert is sent to the ML engineering team to investigate whether model retraining is needed.

Chapter Summary & Key Takeaways

What We Covered

Topic	Key Takeaway
AI Explainability	The discipline of making AI decisions interpretable. Non-negotiable in regulated industries.
LIME	Generates local, model-agnostic explanations by approximating complex models with simple surrogate models near individual predictions.
SHAP	Provides mathematically grounded, globally consistent feature attributions using Shapley values from cooperative game theory.
AI Fairness	Models learn historical biases from training data. Fairlearn and AIF360 provide tools to detect and mitigate demographic disparities.
Observability	The practice of continuously monitoring deployed AI systems for drift, degradation, cost overruns, and safety violations.
Production Integration	All of the above must be integrated into automated pipelines — not run manually — to be effective at enterprise scale.

The Modern AI Engineering Standard

Modern AI systems are no longer judged solely on their predictive accuracy. Stakeholders — users, regulators, auditors, and executives — increasingly require that AI systems also demonstrate that they are:

Accurate • Transparent • Fair • Explainable • Observable • Auditable

The tools and techniques covered in this chapter — LIME, SHAP, Fairlearn, AI Fairness 360, and observability pipelines — are the engineering instruments that allow you to demonstrate these properties with rigour and confidence. Mastering them is not optional for a production-grade AI engineer. It is the baseline expectation.

End of Chapter — Observability & Explainability in AI Systems